Ingegneria del Software — Soluzione del Tema 18/01/2022

Esercizio 1

Si consideri la seguente classe ***immutabile*** Java `OlympicsTable` per la gestione dei risultati di un'edizione dei giochi olimpici. La classe permette di inserire i risultati (`Result`) per ogni disciplina (`Game`) e di consultare informazioni statistiche. Si assuma che ogni gara assegni esattamente una medaglia d'oro, una d'argento e una di bronzo, ovvero non sono possibili casi di parità o medaglie non assegnate.

```
public /*@ pure @*/ class OlympicsTable {
  // Ritorna l'insieme dei risultati.
  public Set<Result> getResults();

  // Aggiunge un risultato (creando e ritornando un nuovo oggetto di tipo OlimpicsTable).
  // Lancia una DuplicateException se e' gia' presente un risultato per
  // la stessa disciplina e per lo stesso sesso.
  public OlympicsTable add(Result res) throws DuplicateException;

  // Ritorna l'elenco dei paesi i cui atleti hanno vinto una medaglia di tipo Medal.
  // L'elenco e' ordinato per numero di medaglie vinte, in ordine decrescente.
  public List<Country> getRank(Medal med);
}

public /*@ pure @*/ class Result {
  // Ritorna la disciplina di questa gara
  public Game getGame();

  // Ritorna il sesso degli atleti partecipanti
  public Sex getSex();

  // Ritorna l'atleta che ha ottenuto la medaglia Medal
  public Athlete getAthlete(Medal med);
  ... }

public /*@ pure @*/ class Athlete {
  // Ritorna la nazionalita' dell'atleta
  public Country getNationality();
  ... }

public enum Medal { GOLD, SILVER, BRONZE }
public enum Sex { F, M }
```

Domanda a)

Si specifichi in JML il metodo `add`.

Soluzione

```
//@requires res != null
//@
//@ensures
//@ !(\exists Result r; getResults().contains(r);
//@     r.getGame().equals(res.getGame()) && r.getSex().equals(res.getSex()) ) &&
//@ \result != null && \result.getResults().contains(res) &&
//@ \result.getResults().size() == getResults().size() + 1 &&
//@ \result.getResults().containsAll(getResults())
//@
//@signals (DuplicateException e)
//@ (\exists Result r; getResults().contains(r);
//@    r.getGame().equals(res.getGame()) && r.getSex().equals(res.getSex()) )
public OlympicsTable add(Result res) throws DuplicateException;
```

Domanda b)

Si specifichi in JML il metodo `getRank`.

Soluzione

```
//@requires med != null
//@
//@ensures \result != null &&
//@ (\forall Country c; ; \result.contains(c) <==>
//@   (\exists Result r; getResults().contains(r);
//@      r.getAthlete(medal).getNationality().equals(c) ) ) &&
//@ (\forall int i; i>=0 && i<\result.size();
//@   (\forall int j; j>=0 && j<i;
//@     !\result.get(i).equals(\result.get(j)) &&
//@     (\num Result r; getResults().contains(r);
//@        r.getAthlete(med).getNationality().equals(\result.get(i))) >=
//@     (\num Result r; getResults().contains(r);
//@        r.getAthlete(med).getNationality().equals(\result.get(j))) ) )
public List<Country> getRank(Medal med);
```

Domanda c)

Si consideri un'implementazione di `OlympicsGame` in cui i risultati sono salvati in due array, uno per le gare femminili e uno per le gare maschili, come mostrato di seguito.

```
public /*@ pure @*/ class OlympicsTable {
  private Result femaleResults[];
  private Result maleResults[];
  ...}
```

Per tale implementazione, fornire l'invariante di rappresentazione (representation invartiant) e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione indica che i due array non sono null, che non contengono risultati nulli o duplicati e che contengono solo risultati per gare femminili (`femaleResults`) e maschili (`maleResults`).

```
private invariant
femaleResults != null && maleResults != null &&
(\forall int i; i>=0 && i<femaleResults.length;
   femaleResults.get(i) != null && femaleResults.get(i).equals(Sex.F) &&
   (\forall int j; j>=0 && j<femaleResults.length;
      femaleResults.get(i).getGame().equals(femaleResults.get(j)) ) ) &&
(\forall int i; i>=0 && i<maleResults.length;
   maleResults.get(i) != null && maleResults.get(i).equals(Sex.M) &&
   (\forall int j; j>=0 && j<femaleResults.length;
      maleResults.get(i).getGame().equals(maleResults.get(j)) ) )
```

La funzione di astrazione si limita a definire `getResults` come unione di tutti e soli gli elementi presenti nei due array. Tutti gli altri metodi pubblici sono infatti definiti a partire da `getResults`.

```
private invariant
(\forall Result r; ; getResults().contains(r) <==>
   (\exists int i; i>=0 && i<femaleResults.length; femaleResults.get(i).equals(r)) ||
   (\exists int i; i>=0 && i<maleResults.length; maleResults.get(i).equals(r)) )
```

Domanda d)

Si consideri una classe `OlympicsGame2` che aggiunge un metodo puro `remove` così definito.

```
// Rimuove un risultato (creando e ritornando un nuovo oggetto di tipo OlimpicsTable).
// Lancia una NoResultException se il risultato non e' presente.
public OlympicsTable remove(Result res) throws DuplicateException;
```

`OlympicsGame2` può essere definita come sottoclasse di `OlympicsGame` in accordo con il principio di sostituzione?

Soluzione

Si, in quanto `OlympicsGame2` non modifica alcun metodo di `OlympicsGame`, ma aggiunge un unico metodo che, essendo puro, non modifica l'istanza su cui è chiamato e non può quindi violare alcuna proprietà pubblica di `OlympicsGame`.

Esercizio 2

Si consideri la seguente classe Java che modella un testo fatto di un numero finito di paragrafi (stringhe immutabili).

```
public class Text {
  private String[] paragraphs;
  public Text(int maxPar) {
    paragraphs = new String[maxPar];
  }
  public synchronized void setPar(int parNum, String par) {
    paragraphs[parNum] = par;
  }
  public void clearPar(int parNum) {
    synchronized(paragraphs) {
      paragraphs[parNum] = null;
    }
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? In caso affermativo motivare la propria risposta. In caso negativo proporre una modifica del solo metodo `setPar` che risolva il problema identificato.

Soluzione

La classe non è correttamente sincronizzata. Il metodo `setPar` è sincronizzato su `this` mentre il metodo `clearPar` è sincronizzato sull'attributo `paragraphs`. Dovremo sincronizzare anche il metodo `setPar` su `paragraphs`, come segue:

```
  public void setPar(int parNum, String par) {
    synchronized(paragraphs) {
      paragraphs[parNum] = par;
    }
  }
```

Domanda b)

Si modifichi il codice del metodo `setPar` affinché sospenda il chiamante se il paragrafo da inserire è già presente (ovvero è diverso da `null`). Si spieghi se e come vadano cambiati gli altri metodi affinchè la sincronizzazione rimanga corretta.

Soluzione

Il metodo `setPar` va modificato come segue:

```
  public void setPar(int parNum, String par) throws InterruptedException {
    synchronized(paragraphs) {
      while(paragraphs[parNum] != null) paragraphs.wait();
      paragraphs[parNum] = par;
    }
  }
```

bisogna inoltre aggiungere l'istruzione: `paragraphs.notifyAll()` all'interno del blocco sincronizzato nel metodo `clearPar`.

Domanda c)

Si aggiunga alla classe `Text` un metodo: `public void print()` che stampi, paragrafo per paragrafo, tutto il testo a terminale, **eseguendo in parallelo rispetto al chiamante**.

Soluzione

```
  public void print() {
    String[] temp = new String[paragraphs.length];
    synchronized(paragraphs) {
      for(int i=0; i<paragraphs.length; i++) temp[i] = paragraphs[i];
    }
```

```
    new Thread() {
      public void run() {
        for(String p : temp) if(p!=null) System.out.println(p);
      }
    }.start();
  }
```

oppure (usando una lambda come corpo del thread):

```
  public void print() {
    String[] temp = new String[paragraphs.length];
    synchronized(paragraphs) {
      for(int i=0; i<paragraphs.length; i++) temp[i] = paragraphs[i];
    }
    new Thread( () -> {for(String p : temp) if(p!=null) System.out.println(p);} ).start();
  }
```

Esercizio 3

Si consideri il programma seguente:

```
1  static void function(int x, int y) {
2    if (x>0) {
3      while (x!=0 && y>0) {
4        y = y - x;
5        x = x - 2;
6        if (x<0) x = -x;
7      }
8    }
9  }
```

Si determini un insieme minimo di casi di test per ciascuno dei casi seguente di copertura.

1. Copertura delle istruzioni (statement coverage)
2. Copertura delle decisioni (edge coverage)
3. Copertura delle decisioni e delle condizioni (edge and condition coverage)

Soluzione

1. Basta un caso di test, p.es. $x = 1, y = 1$, che è quindi anche l'insieme minimo.
2. Il caso precedente non copre le due diramazioni "nascoste" degli if (ovvero i casi in cui gli if vengono valutati negativamente). Servono almeno due casi, in quanto il caso $x = 0$ (y qualunque) copre la diramazione nascosta del primo if, senza eseguire più nulla. Per coprire entrambe le diramazioni del secondo if, occorre che il ciclo sia eseguito almeno due volte. Ad esempio usando $x = 3, y = 4$. L'insieme è quindi minimo.
3. Per coprire tutte le condizioni, occorre coprire l'uscita dal while sia con $x == 0$ che con $y <= 0$. Il caso precedente $x = 3, y = 4$ esce con y negativo; per uscire con $x == 0$ occorre aggiungere un terzo caso: ad esempio $x = 2, y = 3$, in modo che x diventi zero al termine della prima iterazione. Anche questo insieme è quindi minimo.

Esercizio 4

Si consideri il seguente programma Java.

```
public class Oven {
  public void bake(Pizza p) { p.bakedIn(type()); }
  public String type() { return "oven"; }
}

class ExpressOven extends Oven {
  public void bake(CheesePizza p) { p.bakedIn("very " + type()); }
  public void bake(VegetablePizza p) { p.bakedIn("really " + type()); }
  public String type() { return "fast oven"; }
}

abstract class Pizza {
  public void bakedIn(String oven) { System.out.println("Baked in " + oven); }
}

class CheesePizza extends Pizza {
  public void bakedIn(String oven) { System.out.println("Cheese baked in " + oven); }
}

class VegetablePizza extends Pizza {
  public void bakedIn(String oven) { System.out.println("Veg baked in " + oven); }
}
```

Il main è costituito dalle seguenti istruzioni, numerate per riferimento:

```
1  Pizza pizza = new Pizza();
2  CheesePizza pizza1 = new VegetablePizza();
3  Pizza vegPizza = new VegetablePizza();
4  Pizza chsPizza = new CheesePizza();
5  CheesePizza chsPizza1 = new CheesePizza();
6  Oven ov = new Oven();
7  Oven expr = new ExpressOven();
8  ExpressOven expr1 = new ExpressOven();
9  ov.bake(pizza);
10 ov.bake(pizza1);
11 ov.bake(vegPizza);
12 ov.bake(chsPizza);
13 ov.bake(chsPizza1);
14 expr1.bake(vegPizza);
15 expr1.bake(chsPizza);
16 expr1.bake(chsPizza1);
17 expr.bake(vegPizza);
18 expr.bake(chsPizza);
19 expr.bake(chsPizza1);
20 expr.bake(pizza1);
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione. Giustificare brevemente la risposta.

Soluzione

Righe 1 (si istanzia una classe astratta) e 2 (CheesePizza non è sottotipo di VegetablePizza). Di conseguenza, anche 9, 10 e 20.

Domanda b)

Supponendo di eliminare le eventuali righe che generano errori, si scriva, per ogni riga numerata, il valore stampato in uscita.

Soluzione

11: Veg baked in oven
12: Cheese baked in oven
13: Cheese baked in oven
14: Veg baked in fast oven
15: Cheese baked in fast oven
16: Cheese baked in very fast oven
17: Veg baked in fast oven
18: Cheese baked in fast oven
19: Cheese baked in fast oven